Trabajo hecho por:
Miguel Angel Barinas Vanegas
Bryan Steven Beltran Barbosa

Parcial de Patrones de Diseño: Proyecto en Java

Condiciones generales

- **Grupos:** máximo 3 integrantes. No se aceptan grupos de más personas.
- **Lenguaje:** Java.
- **Tipo de aplicación:** ejecutable por consola, con una interfaz gráfica sencilla. Una aplicación web no es obligatoria ni da puntos adicionales.
- **Plazo:** 23 de noviembre de 2026
- **Elección de la propuesta:** cada grupo escoge una de las propuestas de la siguiente sección y la registra con la docente antes de empezar. También puede presentar una propuesta propia, que debe ser aprobada previamente.
- **Repositorio:** el proyecto se entrega en un repositorio Git con historial de commits de todos los integrantes.

Propuestas de software

Las funcionalidades de cada propuesta son opciones, no una lista cerrada. El grupo decide cuáles implementa, las detalla y define el alcance de su sistema. El enunciado no indica qué patrones usar: identificarlos y justificarlos es parte del parcial.

Propuesta 1. Tienda en línea

Sistema de compras con catálogo, carrito y cierre de pedido.

- Productos de varias categorías, cada una con atributos propios (por ejemplo, ropa, tecnología, alimentos).
- Distintos tipos de descuento: por cantidad, por temporada, por cliente frecuente, cupones; algunos acumulables.
- Varios medios de pago simulados, con reglas distintas cada uno (tarjeta, transferencia, contra entrega).
- Extras sobre un producto: garantía extendida, empaque de regalo, envío express.
- Aviso a los clientes interesados cuando un producto agotado vuelve a tener existencias.
- Historial de pedidos y generación de factura.

Propuesta 2. Gestor de tareas y proyectos

Aplicación para organizar el trabajo personal o de un equipo.

- Tareas simples y tareas compuestas por subtareas.
- Estados de una tarea con reglas sobre qué transiciones están permitidas (pendiente, en curso, en revisión, terminada).
- Deshacer y rehacer acciones del usuario.
- Recordatorios y alertas por fecha límite hacia uno o varios responsables.
- Plantillas para crear tareas o proyectos recurrentes.

- Distintas formas de ordenar y filtrar: por prioridad, fecha o responsable.

Propuesta 3. Cafetería o restaurante

Sistema de pedidos para un local con productos personalizables.

- Bebidas y platos con adiciones opcionales que modifican el precio y la descripción.
- Combos armados paso a paso con muchas opciones.
- Distintos tipos de producto creados a partir de un menú configurable.
- Aviso automático a cocina, caja y mesero cuando cambia el estado de un pedido.
- Promociones por horario o por día.
- Cierre de caja y reporte de ventas.

Propuesta 4. Sistema de notificaciones multicanal

Plataforma que envía mensajes por varios canales simulados (correo, SMS, mensajería).

- Cada canal tiene su propio formato y su propia forma de conectarse, como si fueran servicios externos distintos.
- Una sola operación sencilla para que el resto del sistema envíe un mensaje sin conocer los canales.
- Reglas de envío: urgente por todos los canales, informativo solo por uno, reintento si falla.
- Plantillas de mensaje y personalización por usuario.
- Preferencias de cada usuario sobre qué canales acepta.
- Registro de mensajes enviados y fallidos.

Propuesta 5. Biblioteca o préstamo de recursos

Control de préstamos de libros, equipos u otros recursos.

- Varios tipos de recurso con reglas de préstamo diferentes (plazo, cantidad máxima, renovaciones).
- Varios tipos de usuario (estudiante, docente, externo) con privilegios distintos.
- Cálculo de multas con políticas intercambiables.
- Lista de espera: avisar al siguiente usuario cuando un recurso queda disponible.
- Un catálogo único y consistente en toda la aplicación.
- Reportes de uso y de recursos más solicitados.

Qué debe definir cada grupo

La propuesta elegida es un punto de partida. Antes de programar, el grupo debe dejar por escrito lo siguiente.

1. **Alcance:** qué funcionalidades de la propuesta implementará y cuáles dejará fuera, con la razón. Deben ser suficientes para que el sistema tenga al menos tres áreas con variación o cambio frecuente.
2. **Requisitos:** lista de requisitos funcionales y no funcionales del sistema que van a construir.
3. **Problemas de diseño:** al menos tres situaciones del sistema donde un diseño ingenuo causaría código rígido, repetido o difícil de ampliar. Cada una se describe con un ejemplo concreto del proyecto.
4. **Patrones de diseño elegidos:** para cada problema, el patrón que lo resuelve, por qué se eligió frente a otra alternativa y qué consecuencias tiene. Los patrones deben pertenecer a por lo menos dos categorías distintas (creacionales, estructurales o de comportamiento).
5. **Arquitectura general:** cómo se separan la lógica del negocio, la consola y la interfaz gráfica, de modo que ambas interfaces usen el mismo núcleo.

Este plan se presenta a la docente en la primera revisión (ver cronograma) para ajustarlo antes de seguir.

Entregables

1. Software funcional

- Repositorio Git con el código fuente en Java, organizado en paquetes con responsabilidades claras.
- Versión por consola y versión con interfaz gráfica sencilla, ambas sobre el mismo núcleo.
- README con instrucciones para compilar y ejecutar, y la lista de funcionalidades implementadas.
- Pruebas unitarias para las clases donde se aplicó cada patrón.

2. Diagramas UML

- Diagrama de casos de uso del sistema.
- Diagrama de clases general.
- Un diagrama de clases por cada patrón aplicado, con los roles del patrón señalados sobre las clases reales del proyecto.
- Al menos un diagrama de secuencia del flujo principal en el que participe algún patrón.
- Los diagramas deben coincidir con el código entregado.

3. Documento de diseño

Para cada patrón aplicado:

- Problema concreto del proyecto que motivó su uso.
- Descripción del patrón con palabras propias.
- Correspondencia entre los roles del patrón y las clases del proyecto.
- Alternativa que se consideró y por qué se descartó.
- Ventajas, desventajas y relación con los principios SOLID.

Además, una sección final de conclusiones: qué se aprendió y qué se cambiaría del diseño.

4. Sustentación

- Demostración del software y recorrido por el diseño (10 minutos por grupo).
- Preguntas individuales a cada integrante sobre cualquier parte del proyecto.
- La docente puede pedir un cambio de requisito en vivo, por ejemplo agregar una variante nueva, para comprobar que el diseño realmente se puede extender.

Rúbrica de evaluación

La nota del parcial se calcula sobre 100 puntos. Los criterios de la tabla son grupales, salvo la sustentación, que se califica por integrante.

Criterio	Qué se evalúa
Problemas de diseño y elección de patrones	Los problemas son reales y están bien descritos; cada patrón elegido se justifica frente a una alternativa
Aplicación de los patrones en el código	Los patrones están implementados correctamente, resuelven el problema y no son decorativos
Diagramas UML	Notación correcta, roles de cada patrón señalados y coherencia con el código entregado
Documento de diseño	Claridad, explicación con palabras propias, análisis de ventajas y desventajas, relación con SOLID

Software funcional y calidad	Consola e interfaz gráfica sobre el mismo núcleo, pruebas unitarias, código ordenado y repositorio con aportes de todos
Sustentación individual	Dominio del diseño, respuesta a preguntas y capacidad de extender el sistema ante un cambio de requisito

Reglas de integridad académica

- Cada integrante debe tener commits propios en el repositorio. Un aporte nulo o mínimo se reflejará en su nota individual.
- El trabajo debe ser original del grupo. Si se usa código, diagramas o texto de otras fuentes, incluidas herramientas de inteligencia artificial, se debe declarar cuál y para qué se usó, y el grupo debe poder explicarlo.
- Si en la sustentación un integrante no puede explicar una parte del proyecto, su nota individual se ajusta.
- Proyectos idénticos o muy similares entre grupos se calificarán como no originales.
- Los grupos de más de 3 personas no serán calificados.

Propuesta 5. Biblioteca y préstamo de recursos

1. Alcance

El sistema controlará el préstamo, la devolución y la renovación de recursos (libros, equipos y otros) sobre un catálogo único y consistente, e implementará tres áreas de variación con seguimiento: reglas de préstamo diferentes según el tipo de recurso, cálculo de multas con políticas intercambiables y lista de espera con aviso al siguiente usuario, donde quedan fuera los tipos de usuario con privilegios distintos, los reportes de uso, los pagos en línea, la adquisición de recursos, el envío real de correos o SMS y la integración con sistemas externos, por no aportar a las áreas de variación o exceder el alcance académico.

2. Requisitos

2.1 Requisitos funcionales

ID	Requisito
RF-01	El sistema debe permitir registrar, consultar, modificar y dar de baja recursos de distintos tipos en un catálogo único
RF-02	El sistema debe permitir registrar el préstamo de un recurso a un usuario, validando su disponibilidad y las reglas de su tipo de recurso
RF-03	El sistema debe permitir registrar la devolución de un recurso y actualizar su disponibilidad
RF-04	El sistema debe permitir renovar un préstamo, respetando el número máximo de renovaciones

RF-05	El sistema debe calcular automáticamente la multa por retraso
RF-06	El sistema debe avisar al siguiente usuario de la lista cuando el recurso sea devuelto y quede disponible.

2.2 Requisitos no funcionales

ID	Requisito
RNF-01	El sistema debe procesar transacciones críticas en un tiempo de respuesta no mayor a 2 segundos bajo condiciones normales de operación.
RNF-02	El catálogo único debe soportar la consulta concurrente de al menos 500 usuarios sin que se degrade el rendimiento de la aplicación
RNF-03	El sistema debe implementar un control de acceso basado en roles (RBAC) para garantizar que los privilegios de cada tipo de usuario y administrador se apliquen de forma estricta.
RNF-04	Las contraseñas y datos personales sensibles de los usuarios deben almacenarse de manera cifrada (por ejemplo, utilizando algoritmos como bcrypt) en la base de datos.
RNF-05	El sistema debe implementar el patrón de diseño *Strategy* en el módulo de cálculo de multas, permitiendo que las políticas de penalización sean intercambiables sin modificar la lógica central del código.
RNF-06	El software debe desarrollarse bajo una arquitectura de capas, utilizando patrones como DAO y DTO, para asegurar un acoplamiento bajo y facilitar el mantenimiento del catálogo y los reportes.

REPOSITORIO: https://github.com/MiguelAngel-cmd/Proyecto-Ing-Software-2.git